Step 1: Formalize the Module's Architectural Position

Begin by defining where SIFT sits in the "Cybersecurity Mitigation Maturity Model." Research suggests classifying your tool across five levels: Ad hoc, Planned, Standardized, Metrics-Driven, or Continuous Improvements.

- **Action:** Document SIFT's architecture (e.g., is it a RAG-enhanced system, a Dual-Agent framework, or a Chain-of-Thought reasoning model?).
- **Validation:** Use the "System Security Plan" approach to define the system boundary, operational environment, and how security requirements are implemented.

Step 2: Conduct a Targeted Literature Comparison

Position SIFT against the "vulnerability discovery and remediation" benchmarks established by DARPA's AI Cyber Challenge (AIxCC 2023–2025).

- **Literature Focus:** Contrast SIFT with recent studies on AI-induced supply-chain risks, specifically **slopsquatting** (the registration of hallucinated package names).
- **Gap Identification:** Highlight how traditional SAST tools often miss context-dependent logic errors that SIFT's AI-driven approach is designed to catch.

Step 3: Configure the Evaluation Environment

To ensure your paper meets academic standards, move beyond basic code snippets and use repository-level datasets.

- **Datasets:** Utilize the **A.S.E (AI Code Generation Security Evaluation)** benchmark or the **OpenSSF CVE Benchmark**, which has been recently updated to support multi-line alerts—crucial for evaluating the deep reasoning of AI auditors.
- **Benchmark Setup:** Containerize your testing environment using Docker to ensure reproducibility, as recommended for modern cybersecurity research.

Step 4: Execute Quantitative Benchmarking

Replace the standard F1 score with a "cost-aware" metric to reflect real-world security implications.

- **Metric (Cscore):** Use the Cscore to account for the fact that a false negative (missed vulnerability) is significantly more expensive than a false positive (wasted developer time). The formula is:

$$\text{Cscore} = \left(\frac{1}{\text{Precision}} - 1 - r_c \right) \cdot \text{Recall} + r_c$$

- **Performance Metrics:** Measure SIFT's success in detecting synthetic vulnerabilities (aim for the 2025 benchmark of 86%) and its successful patching rate (benchmark is 68%).

Step 5: Evaluate Reasoning and Explainability

Since you are presenting SIFT as a "code auditor," you must evaluate the quality of its advice, not just its binary detection.

- **Chain-of-Thought (CoT) Analysis:** Evaluate if SIFT follows data and control dependencies. Research shows that CoT-inspired prompting can lead to a 553.3% higher F1 accuracy in vulnerability identification compared to basic prompts.

- **Monitorability:** Apply the **Monitor Approval Rate (Am)** metric to determine if a human or a secondary "monitor model" finds SIFT's internal reasoning legible and faithful.

- **XAI Metrics:** Specifically measure **Recovery Rate** (trigger detection effectiveness) and **Fidelity** (how closely the explanation matches the actual model decision).